

$\Rightarrow (u,v)$ is called a safe edge

Generic - MST (G, w)

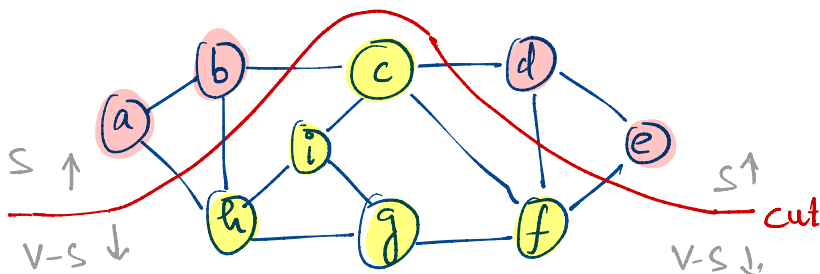
$A = \emptyset$

While A does not form a spanning tree

 | find an edge (u,v) that is safe for A
 | $A = A \cup \{(u,v)\}$

return A

Definition:



A cut $(S, V-S)$ of an undirected graph $G = (V, E)$ is a partition of V . An edge $(u,v) \in E$ crosses the cut $(S, V-S)$ if one of its end-points belongs to S and the other to $V-S$.

- A cut is said to respect a set A of edges if no edge in A crosses the cut.
- An edge is a light edge crossing a cut if its weight is minimum of any edge cutting the cut.

Kruskal's algorithm

- Uses a disjoint-set data structure to maintain several disjoint set of elements.
- Each set contains the vertices in one tree of the current forest
- Find-Set(u) returns a representative element from the set that contains u
 - To check if two vertices u & v belong to the same tree, we just need to test if $\text{Find-Set}(u) = \text{Find-Set}(v)$
- Union(u, v) procedure combines two trees containing u & v .

MST-Kruskal(G, w)

$A = \emptyset$

For each vertex $v \in G.V$

Make-Set(v) // create $|V|$ trees

Create a single list of edges in $G.E$

Sort the list of edges into monotonically increasing order by weight w

For each edge (u, v) taken from the sorted list in order

if $\text{Find-Set}(u) \neq \text{Find-Set}(v)$

$A = A \cup \{(u, v)\}$ // add edge (u, v) to MST

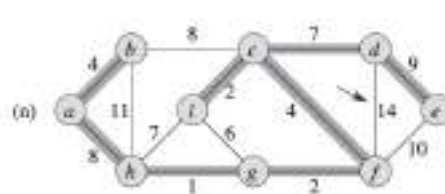
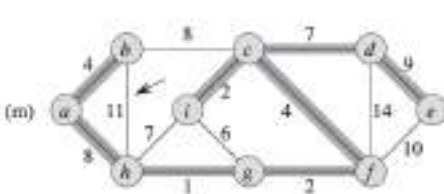
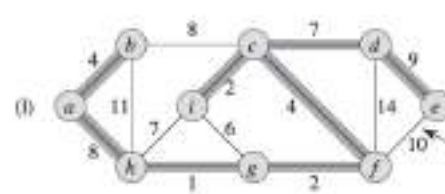
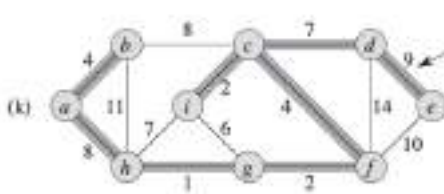
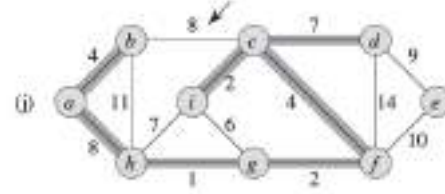
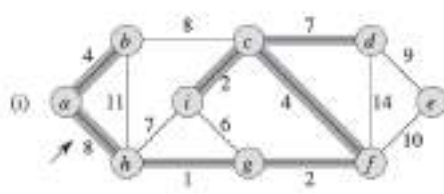
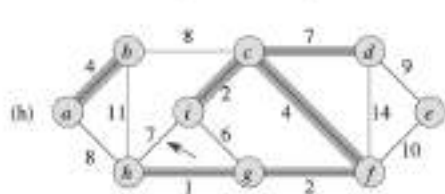
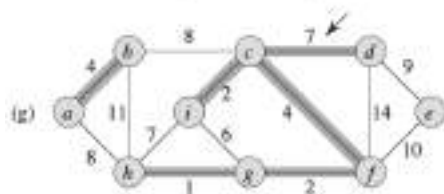
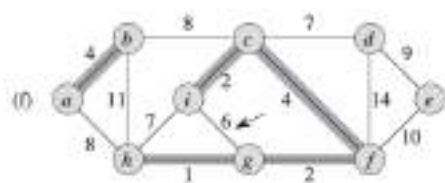
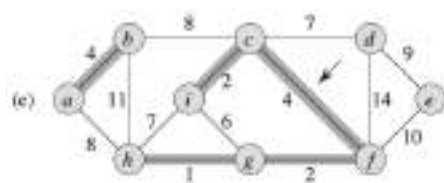
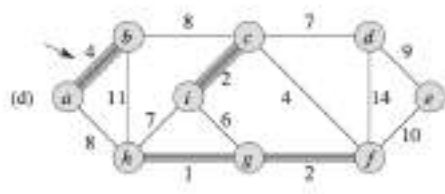
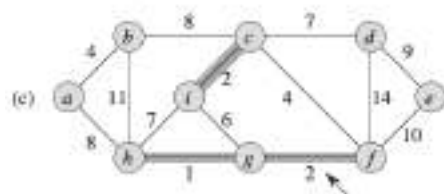
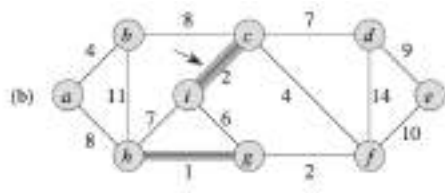
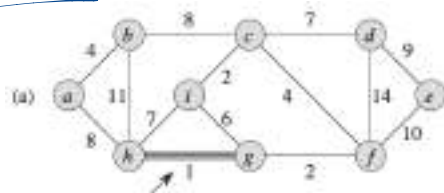
Union(u, v)

// Merge trees of u & v

return A

$O(|E| \lg |V|)$

Example 3



Prim's algorithm

- Maintains a single tree A during execution;
Stops when A spans all vertices in V .
- Each step adds to A , a light edge that connects A to an isolated vertex

one on which no edge of A is incident
- This rule adds only edges that are safe for A .
- The algorithm maintains a min-priority queue Q of all vertices that are not in the tree based on a key attribute.
- For each vertex v , the attribute $v.key$ is the minimum weight in the tree, where by convention $v.key = \infty$ if there is no such edge.

MST - PRIM(G, w, r) : r - an arbitrary root

for each vertex $u \in G.V$

| $u.key = \infty$
| $u.\pi = NIL$

$r.key = 0$

$Q = \emptyset$

For each vertex $u \in G.V$

Insert(Q, u)

While $Q \neq \emptyset$

$u = \text{Extract-Min}(Q)$ // add u to MST

for each vertex v in $G.Adj[u]$

| if $v \in Q$ and $w(u, v) < v.key$

| $v.\pi = u$

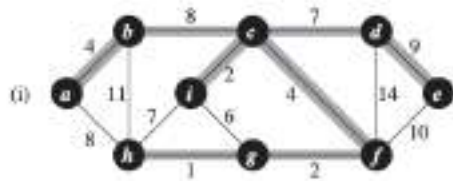
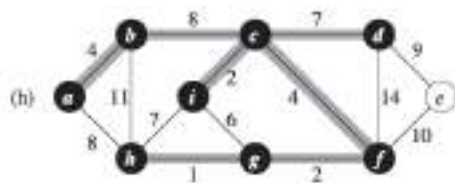
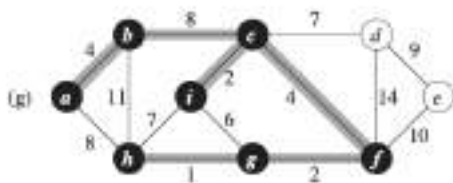
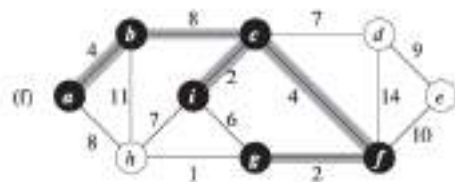
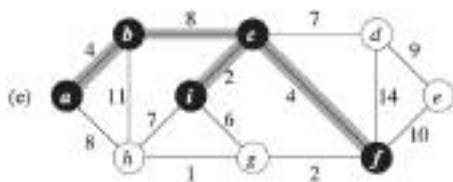
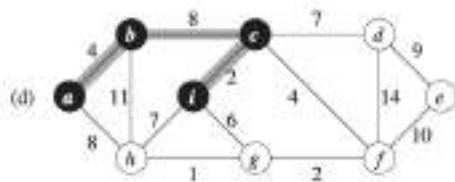
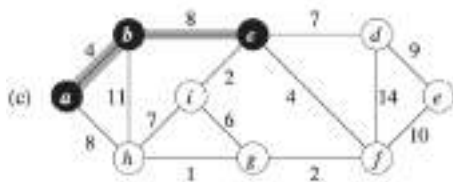
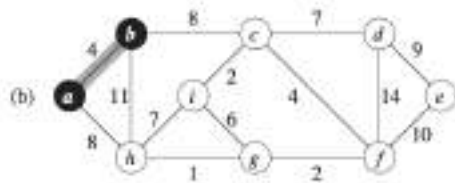
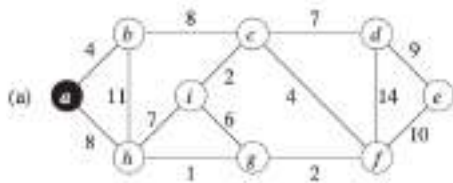
| $v.key = w(u, v)$

| Decrease-key($Q, v, w(u, v)$)

// update keys
of u 's non-tree
neighbours!

$O(E \lg V)$

Example 3

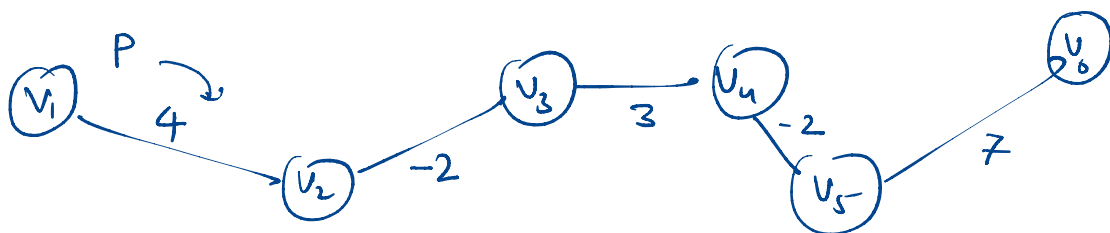


Shortest Paths

- Consider a digraph $G = (V, E)$ with edge weight function $w: E \rightarrow \mathbb{R}$,

- The weight of a path $P = \langle v_0, v_1, \dots, v_k \rangle$ is defined as

$$W(P) = \sum_{i=0}^k w(v_i, v_{i+1})$$



$$W(P) = 10$$

Shortest path: A shortest path from u to v is a path of minimum weight from u to v .

The shortest path weight is defined as:

$$\delta(u, v) = \begin{cases} \min_{\infty} \{ w(P) : u \xrightarrow{P} v \} & \text{if there is a path from } u \text{ to } v, \\ \text{otherwise} \end{cases}$$

Single Source Shortest Path Problem:

Given a graph $G=(V,E)$, find the shortest-path weights $\delta(s,v)$ for all $v \in V$ from a given source vertex $s \in V$

→ results in a shortest path tree rooted at s .

Optimal Substructure Property :

Theorem: A subpath of a shortest path is a shortest path.

formally: Given a weighted directed graph $G=(V,E)$ with weight function $w:E \rightarrow \mathbb{R}$, let

$P = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from v_0 to v_k .

For any i, j such that $0 \leq i \leq j \leq k$, let

$P_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be a subpath of P from v_i to v_j , then P_{ij} is a shortest path from v_i to v_j .

↓

Proof: Decompose path p into:

$$v_0 \xrightarrow{P_{0i}} v_i \xrightarrow{P_{ij}} v_j \xrightarrow{P_{jk}} v_k, \text{ so}$$

that

$$w(p) = w(P_{0i}) + w(P_{ij}) + w(P_{jk}).$$

Let's assume that there exists a better path (lower weight) P'_{ij} from i to j with smaller weight $w(P'_{ij}) < w(P_{ij})$. Then

$v_0 \xrightarrow{P_{0i}} v_i \xrightarrow{P'_{ij}} v_j \xrightarrow{P_{jk}} v_k$ is a path from v_0 to v_k whose weight $w(P_{0i}) + w(P'_{ij}) + w(P_{jk})$ is less than $w(p)$, which contradicts our assumption that p is the shortest path between v_0 & v_k .



Theorem: Triangle inequality

Let $G=(V,E)$ be a weighted directed graph with weight function $w: E \rightarrow \mathbb{R}$ and a source vertex s , then for all edges $(u,v) \in E$,

$$\delta(s,v) \leq \delta(s,u) + w(u,v)$$

Proof: Suppose p is shortest path from s to u , then $\delta(p)$ has the least weight for any path from s to u . In particular this is also true for path s to u & then u to v .



- Negative weight cycles

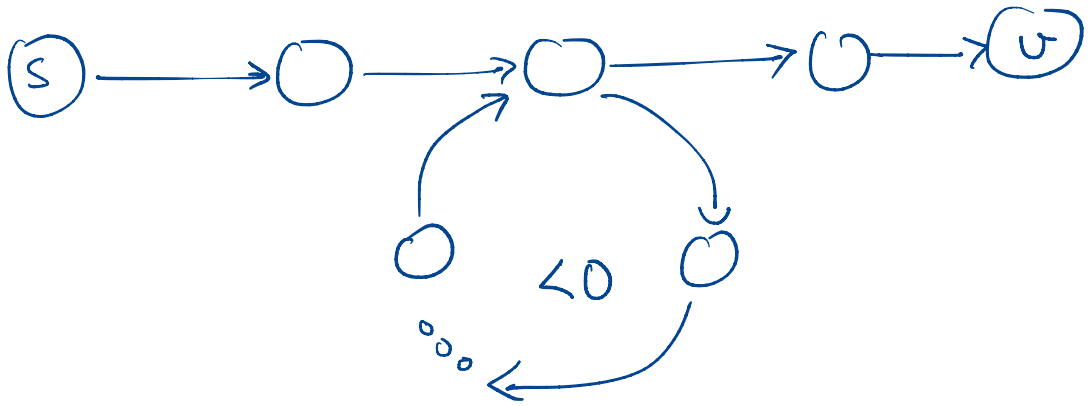
If a graph G contains negative weight cycles, then some shortest paths may not exist.

⇒ If G contains no negative weight cycle reachable from source s , then for all $v \in V$, the weight $\delta(s, v)$ remains well defined.

→ On the contrary if G contains a -ve weight cycle reachable from s , then shortest path weights are not well defined

↓
i.e., one can always find a path with lower weight by traversing the

negative weight cycle.



- Can a shortest path contain a cycle?

NO

- Negative - weight cycles are certainly not possible

→ Positive - weight cycles will imply that one can always remove them to obtain a path of smaller weight.

→ Zero - weight cycles can also be removed with no consequence.

Greedy approach:

- maintain a set T of vertices whose shortest path distances from vertex s are known
- At each step add to T the vertex $v \in V - T$ whose distance estimate from s is minimum
- Update distance estimates of vertices adjacent to v .

Dijkstra's algorithm ?

Directed graph $G = (V, E)$, non-negative weights $w(u, v) \geq 0$ for each edge $(u, v) \in E$

↳ Generalizes BFS to weighted graphs.

T : Set of vertices whose final shortest path weights from s have been determined

Q : min priority queue of vertices on d

DIJKSTRA(G, w, s)

for each vertex $v \in G.V$

$v.d = \infty$ $\rightarrow$ distance estimate from s to v .
 $v.\pi = \text{NIL}$

$s.d = 0$

$T = \emptyset$

$Q = \emptyset$

for each vertex $v \in G.V$

 Insert(Q, v)

While $Q \neq \emptyset$

$u = \text{Extract-Min}(Q)$

$T = T \cup \{u\}$

 for each vertex v in $G.\text{Adj}[u]$

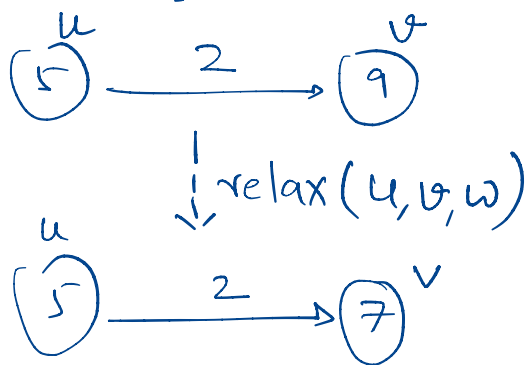
Relaxation Step: $\left\{ \begin{array}{l} \text{if } v.d > u.d + w(u, v) \\ \quad v.d = u.d + w(u, v) \\ \quad v.\pi = u \\ \quad \text{Decrease-Key}(Q, v, v.d) \end{array} \right.$

Relaxation step: tests if going through

vertex u improves the shortest path to v found so far.

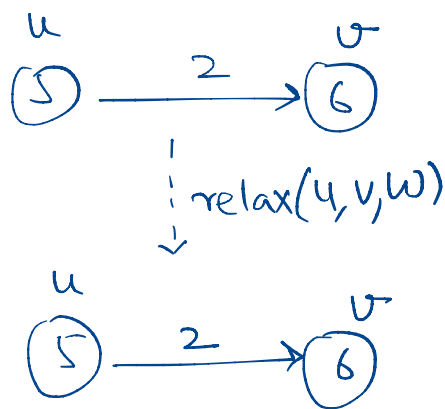
If so, update $v.d$ & v 's parent.

Example of relaxation:



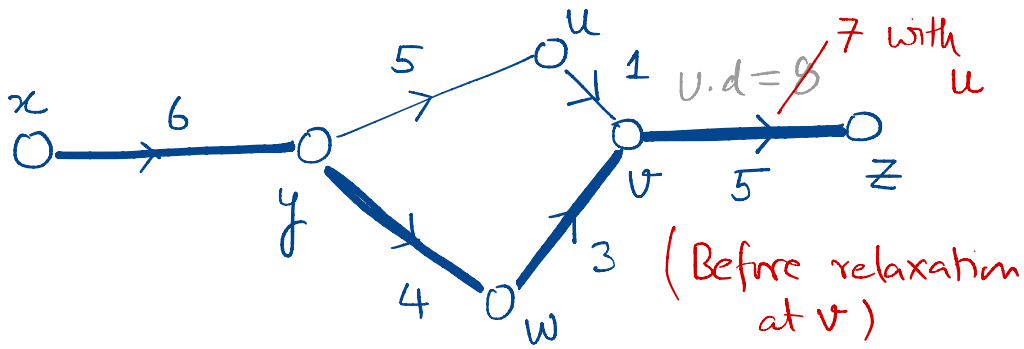
case I

(relaxation reduces $v.d$)

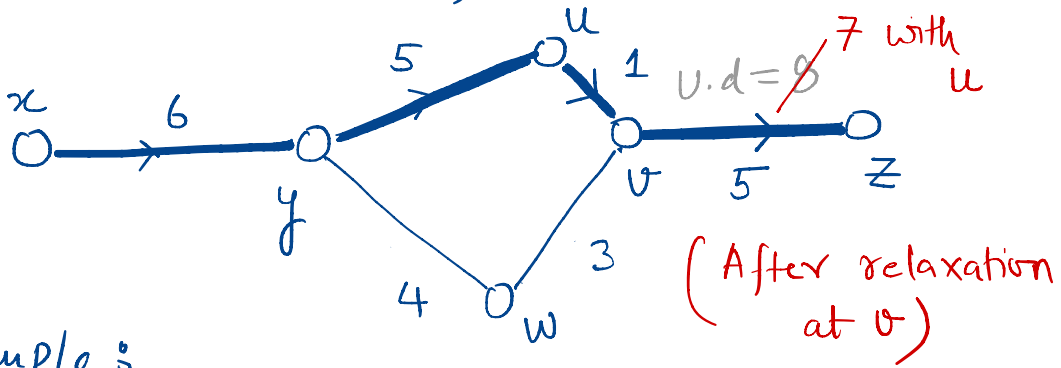


case II

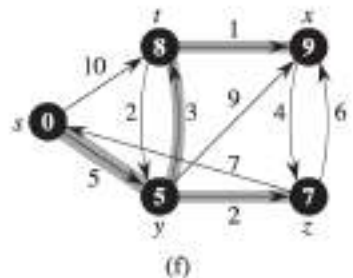
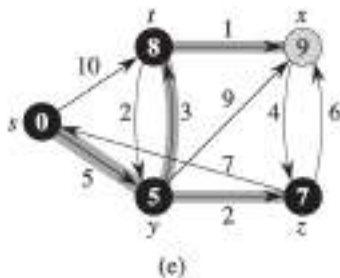
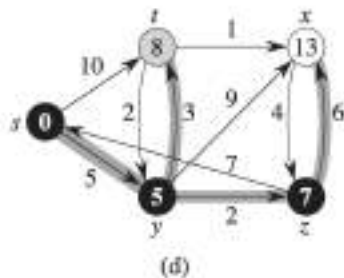
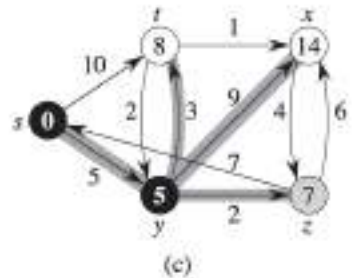
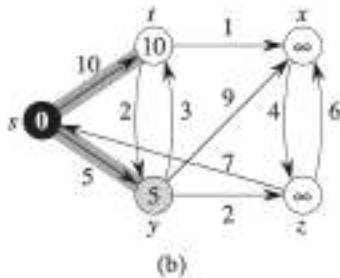
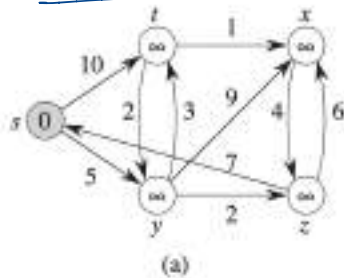
(relaxation has no effect on $v.d$)



Path $P \langle x, y, w, v, z \rangle$ can be relaxed to
 Path $P' \langle x, y, u, v, z \rangle$ of lower distance



Example :



Time Complexity of Dijkstra with min-priority queue using a min-heap:

Insert $\rightarrow O(V)$ // each vertex inserted once

Extract-Min $\rightarrow O\left(\sum_{i=1}^V \lg(i)\right) \quad O(\lg(V!))$

$= O(V \lg V)$ [use Sterling's approx.]
// each vertex extracted once.

Decrease-key $\rightarrow$ at most $|E|$ operations
each taking $O(\lg V)$ time
 $= O(E \lg V)$

Total complexity = $O((V+E) \lg V)$

In a typical case when $|E| = \Omega(V)$,
complexity = $O(E \lg V)$